EnduranceLife API

Technical Report: Design Choices, Architecture & Reflections

David Feng

April 2026

1 Introduction

EnduranceLife is a RESTful Web Service API designed for managing endurance-sport training data. It ingests workout recordings from Coros GPS watches (via `.fit` files), cross-references them with daily lifestyle metrics (sleep, nutrition, fatigue) and physiological snapshots (VO2Max, resting heart rate), and exposes five analytics endpoints that produce chart-ready JSON for front-end dashboards.

The service is deployed on Render.com with a managed PostgreSQL database, while retaining full local-development compatibility via SQLite.

2 Technology Stack & Justification

2.1 Programming Language: Python 3.11

Python was chosen for three reasons:

1. **Ecosystem fit.** The `fitdecode` library — the only decoder that tolerates Coros's non-standard FIT field sizing — is a Python package. Using Python eliminates the need for cross-language interop.
2. **Rapid prototyping.** Dynamic typing and a concise syntax accelerated development of the four data-pipeline scripts and the analytics endpoint logic.
3. **Data-science readiness.** Future extensions (e.g. ML-based race prediction, anomaly detection) can leverage Python's NumPy / scikit-learn ecosystem without rewriting the service.

2.2 Framework: FastAPI

FastAPI was selected over Flask and Django REST Framework (DRF):

Criterion	**FastAPI**	**Alternatives**
Auto-generated docs	Swagger + ReDoc built-in	Flask: manual; DRF: partial
Type validation	Pydantic V2 (native)	Flask: add-on; DRF: serializers
Async support	First-class ASGI	Flask: WSGI; DRF: partial
Performance	Comparable to Go/Node	Flask/DRF notably slower
Learning curve	Minimal boilerplate	DRF: steeper

Table 1: Framework comparison matrix

FastAPI's automatic OpenAPI schema generation was particularly valuable: every endpoint is immediately testable via the `/docs` Swagger UI and documented via `/redoc`, satisfying the project's requirement for a self-documenting API with zero additional tooling.

2.3 ORM: SQLAlchemy 2.0

SQLAlchemy provides database-agnostic ORM mapping, enabling the same Python model classes to work against both SQLite (local development and testing) and PostgreSQL (production on Render). This dual-backend strategy was implemented with a single environment-variable check in `database.py`.

2.4 Database: PostgreSQL (Production) / SQLite (Development)

- **PostgreSQL** was chosen for production because Render offers a managed free-tier instance, it is a fully relational SQL database satisfying the coursework requirement, and it supports concurrent connections from the Uvicorn worker processes.
- **SQLite** is used locally and in tests for zero-configuration development. The in-memory SQLite backend makes the 66-test suite execute in under 2 seconds with complete isolation.
- **Why not NoSQL?** The data model is inherently relational: activities reference a user (`pid`), daily metrics are keyed by `(pid, date)`, and analytics queries rely on SQL aggregations (`GROUP BY`, `SUM`, `MIN`). A document store would complicate these cross-entity joins.

2.5 Deployment: Render.com

Render was chosen for its GitHub-integrated CI/CD pipeline, free PostgreSQL tier, and automatic HTTPS. The deployment requires only two configuration values:

```
Build Command:  pip install -r requirements.txt
Start Command:  uvicorn app.main:app --host 0.0.0.0 --port $PORT
```

A `.python-version` file pins the runtime to Python 3.11.7 to ensure pre-built wheels are available for `pydantic-core` (which requires Rust compilation on unsupported versions).

3 Architecture & Design Decisions

3.1 Project Structure

The codebase follows a **domain-driven router pattern**:

```
app/
  main.py            # Entry point, router registration
  database.py        # Engine + session factory (PostgreSQL / SQLite)
  models.py          # SQLAlchemy ORM (User, Activity, DailyMetric, ...)
  schemas.py         # Pydantic request/response models
  auth.py            # JWT utilities (bcrypt hashing, token create/verify
     )
  routers/
    auth.py          # Register, login, logout, /me
    activity.py      # CRUD + .fit upload
    daily_metric.py  # CRUD + composite-key update
    physiology.py    # CRUD for physiological snapshots
    analytics.py     # 5 dashboard-facing endpoints
```

Each router owns a single URL prefix and tag, keeping concerns isolated and enabling independent testing.

3.2 Data Pipeline Architecture & API Endpoints

The core data ingestion is handled through endpoints in the API and four standalone ETL scripts in `scripts/`:

1. **File Uploads & FIT Parsing (`/activities/upload`):** Rather than relying entirely on manual JSON entry, the API accepts binary `.fit` files (the standard used by Garmin, Coros, Wahoo) via `multipart/form-data`. The upload endpoint delegates parsing to the `fitdecode` library. It iterates through the file frames to extract bounding timestamps (start, end), total distance, elevation ascent/descent, and computes metrics like average pace or heart rate dynamically from the data stream. Finally, these parsed metrics are automatically mapped to the SQLAlchemy `Activity` ORM and inserted into the PostgreSQL database.

2. **RESTful Pagination & Filtering (`/activities/` & `/daily-metrics/`):** The list endpoints utilize query parameters (`skip`, `limit`, `date_from`, `date_to`, `pid`) to dynamically construct SQLAlchemy '.filter()' clauses. This pushes the filtering down to the database engine level (e.g., executing `WHERE user_id = 1 AND start_time >= '2025-01-01'`), preventing memory bloat by returning only the paginated slice of records requested.

3. **Composite-Key Updates (`PUT /daily-metrics/by-date`):** The Daily Metrics router implements an upsert/update strategy based on a composite business key (`pid` + `date`). This allows smart integrations (or frontend clients) to send a payload for "today's stats" without needing to perform a prior `GET` request to look up the database's internal row ID, significantly reducing network overhead.

4. **Data Enrichment Pipeline:** Supplemental standalone scripts execute batched background operations. **import_fit.py** handles directory-wide ingestion. **enrich_weather.py** calls the external Open-Meteo API using latitude/longitude extracted from activities to backfill missing environmental data. **seed_daily_metrics.py** & **seed_physiology.py** generate correlated simulated lifestyle data and physiological progressions (using sine-wave trending and randomized fatigue noise) to provide realistic sample testing datasets.

All scripts use **batch commits** (configurable, default 30–50 records per flush) and **in-memory duplicate detection** (pre-loading existing keys into a `set()`) to minimize network round-trips when operating against a remote PostgreSQL instance.

3.3 Analytics Engine

The five analytics endpoints (`/analytics/*`) are designed as **complex SQL aggregations** routed directly into Pydantic response models. By offloading computation to the database layer (PostgreSQL), the API minimizes memory usage. Key implementations include:

- **Race Predictor / Physiology Trends:** Uses the Daniels VO2-velocity quadratic formula ($\text{VO2} = -4.60 + 0.182258v + 0.000104v^2$) iteratively solved or derived. The algorithm is mapped in Python to convert the user's latest recorded `vo2_max` logarithmically into sustainable pace forecasts for 5K, 10K, and Half Marathons. It uses calibrated sustainability factors (e.g., humans cannot indefinitely hold VO2Max; 5K pace operates at 95% VO2Max, HM at 85%).

- **Performance Records (PR Filtering Engine):** Tracks fixed-distance best times by grouping and ranking activities. Crucially, the logic involves rigorous **anomaly filtering**. GPS errors frequently record impossible pacings. The logic excludes data outside physiologically plausible bounds ($<$ 2:30/km or $>$ 10:00/km for runs; $<$ 8 km/h or $>$ 70 km/h for rides).

- **Environment Insights (Temperature Bucketing):** Utilizing SQLAlchemy's Conditional

Expressions (`case` statements), weather data is bucketed dynamically. The query groups activities into three thermal constraints: Cold ($< 10°C$), Moderate ($10 - 22°C$), and Hot ($> 22°C$). Averages for heart rate and speed are then computed *per bucket* via `func.avg()`, mathematically illustrating the exact cardiovascular drift caused by extreme heat.

- **Lifestyle Impact (Cross-Table Joins):** Correlates the `daily_metrics` table (subjective fatigue, sleep hours) with the `activities` table (objective pace, heart rate). Utilizing SQL `JOIN` clauses on `date(start_time) == date`, the endpoint categorizes performance against sleep states (e.g., comparing days with >7h sleep vs <6h sleep) to expose behavioral trends.

3.4 Dual-Database Strategy

A single three-line conditional in `database.py` enables seamless switching:

```
DATABASE_URL = os.environ.get("DATABASE_URL")
if DATABASE_URL:
    # Fix Render's postgres:// -> postgresql://
    engine = create_engine(DATABASE_URL)
else:
    engine = create_engine("sqlite:///./endurance_life.db",
                           connect_args={"check_same_thread": False})
```

This pattern requires zero code changes between local development, CI testing (in-memory SQLite), and production deployment (Render PostgreSQL).

3.5 Authentication

The API implements JWT-based authentication with four endpoints:

- `POST /auth/register` — Create a new user (bcrypt-hashed password storage).
- `POST /auth/login` — Verify credentials and return a signed JWT (30 min expiry, configurable via `TOKEN_EXPIRE_MINUTES`).
- `POST /auth/logout` — Invalidate the token by blacklisting its unique `jti` (JWT ID) claim.
- `GET /auth/me` — Return the authenticated user's profile.

Key design choices:

- **Global Data Isolation (IDOR Prevention):** To prevent Insecure Direct Object Reference vulnerabilities, the `pid` field is stripped from all client payloads and query parameters. Instead, the `get_current_user` FastAPI dependency uses token extraction to tightly scope all SQL CRUD operations and Analytics aggregations exclusively to the logged-in user's ID (`current_user.id`).
- Each JWT contains a UUID-based `jti` claim, ensuring that tokens issued in the same second are distinguishable and can be blacklisted independently.
- The `get_current_user` dependency is injected into all 23 protected routes to enforce authentication synchronously.
- The blacklist is in-memory (a Python `set`); in production, this would be replaced with Redis or a database table for persistence across restarts.
- **Pre-seeded Demo User.** To facilitate evaluating the populated dashboard analytics data smoothly, an application startup hook automatically seeds a default user (`username: demo`, `password: endurance2026`) as user `id=1` if the database is empty. All generated mock data routes to this `pid=1`. *(Testing Note: Evaluators should use the global "Authorize" button at*

the top of the Swagger UI rather than executing the `/auth/login` block manually. The global button natively extracts the stateless JWT from the HTTP 200 response and seamlessly injects it into all future requests, accurately simulating a frontend client state.)

4 Testing Approach

The project maintains an **80-test suite** using `pytest`, organized into five test modules:

Module	Scope	Tests
`test_auth.py`	Register, login, logout, token validation	14
`test_activity.py`	CRUD + .fit upload + filters + pagination	19
`test_daily_metric.py`	CRUD + composite-key update + validation	14
`test_physiology.py`	CRUD + JSON zone handling	13
`test_analytics.py`	All 5 analytics endpoints	20

Table 2: Test suite breakdown

Key design decisions:

- **In-memory SQLite** — Every test session spins up a fresh `sqlite:///:memory:` database via a pytest fixture, ensuring complete isolation and sub-2-second execution.
- **Fixture composition** — `conftest.py` provides layered fixtures (`db_session` → `client` → `sample_activity` → `analytics_data`) that build on each other, reducing boilerplate.
- **Edge-case coverage** — Tests verify 409 on duplicates, 422 on validation failures, 404 on missing resources, anomaly filtering thresholds, and distance-band matching for PR records.

5 Challenges & Lessons Learned

5.1 Coros FIT File Parsing

Coros watches produce non-standard FIT files where fields declared as `uint32` are packed into fewer bytes. The popular `fitparse` library crashes on these files with `FitParseError`. Switching to `fitdecode` — which emits warnings instead of exceptions — resolved this entirely. **Lesson:** when dealing with hardware vendor data, always test with real device exports, not just synthetic test files.

5.2 Route Ordering in FastAPI

A subtle bug emerged where `PUT /daily-metrics/by-date` was being captured by the `PUT /daily-metrics/{metric_id}` route, because FastAPI matches routes in declaration order and parsed `"by-date"` as a path parameter. Moving the `/by-date` route *before* the `/{metric_id}` route fixed the issue. **Lesson:** in path-parameter-heavy APIs, always declare specific literal paths before parameterized ones.

5.3 Remote Database Performance

Initial data-pipeline scripts committed every record individually, resulting in 1430+ network round-trips across the Pacific to Render's Oregon-based PostgreSQL. This took over 10 minutes. Refactoring to batch commits (30–50 records per flush) with in-memory duplicate detection reduced the import time to under 3 minutes. **Lesson:** database access patterns that are invisible with local SQLite become critical bottlenecks with remote databases.

5.4 Python Version Pinning

Render defaulted to Python 3.14, which lacks pre-built wheels for `pydantic-core`. The resulting Rust compilation failed on Render's read-only filesystem. Adding a `.python-version` file pinning to 3.11.7 resolved this. **Lesson:** always pin your runtime version for cloud deployments.

5.5 CDN Accessibility

FastAPI's default Swagger UI and ReDoc pages load JavaScript from `cdn.jsdelivr.net`, which is blocked in mainland China. A custom `/redoc` endpoint serving the script from `unpkg.com` was implemented as a workaround. **Lesson:** CDN availability varies by region; self-hosting or alternative CDNs may be necessary.

6 Limitations & Future Work

6.1 Current Limitations

- **In-memory token blacklist.** The current logout implementation stores invalidated JTIs in a Python `set`, which is lost on server restart. A Redis-backed blacklist would provide persistence.
- **Schema migrations.** Tables are created via `Base.metadata.create_all()`, which cannot handle column additions or renames. Alembic should be adopted for schema versioning.
- **No per-user data isolation.** Authentication is implemented, but CRUD endpoints do not yet enforce that users can only access their own data. Adding `current_user.id == activity.pid` guards would complete the authorization layer.
- **Simulated lifestyle data.** The `seed_daily_metrics` and `seed_physiology` scripts generate mock data. A real integration would import from wearable APIs (e.g. Coros Health SDK, Apple HealthKit).
- **Render free-tier constraints.** The PostgreSQL instance is limited to 1 GB and may be recycled after 90 days of inactivity.

6.2 Potential Improvements

- **Real-time sync** — Implement OAuth-based integration with Coros / Garmin Connect to auto-import new activities.
- **ML race predictor** — Replace the formula-based predictor with a regression model trained on the user's actual race results.
- **Frontend dashboard** — Build a Vue.js or React SPA consuming the analytics endpoints, with ECharts for interactive charts.
- **WebSocket notifications** — Push real-time updates when new activities are imported or weather data is enriched.
- **Containerization** — Dockerize the application for reproducible deployments across cloud providers.

7 Generative AI Declaration

7.1 Declaration of Use

Generative AI tools were used during the development of this project. Specifically, **Google Gemini (via Antigravity IDE)** was used as a pair-programming assistant throughout the development process.

7.2 How AI Was Used

1. **Code generation.** AI generated initial implementations of CRUD routers, Pydantic schemas, and test fixtures. Every generated block was reviewed, tested, and frequently modified to fit the project's conventions.
2. **Algorithm calibration.** The Daniels VO2-velocity formula implementation was iteratively refined with AI assistance, cross-referencing the mathematical model against my real-world 5K and half-marathon performance data to calibrate sustainability factors.
3. **Debugging.** AI identified the FastAPI route-ordering conflict and the Render Python version incompatibility, both of which would have required significant manual investigation.
4. **Performance optimization.** The batch-commit refactoring of all four data-pipeline scripts was developed collaboratively with AI, discussing trade-offs between memory usage and network round-trips.
5. **Documentation.** This report's structure was drafted with AI assistance, though all technical content reflects my actual design decisions and real development experience.

7.3 Critical Reflection

AI accelerated development significantly — the 80-test suite and five analytics endpoints were built in a fraction of the time manual development would have required. However, AI assistance also introduced risks:

- **Over-reliance on suggestions.** The initial race prediction formula generated by AI was mathematically incorrect (a simplistic linear model). It required manual verification against published sports-science research (Daniels' Running Formula) and calibration against real data to produce accurate predictions.
- **Platform-specific blindspots.** AI initially used Unicode box-drawing characters in print statements that crashed on Windows' CP1252 terminal encoding. Real-world testing caught this.
- **Shallow understanding.** AI suggested the `StandaloneLayout` preset for Swagger UI without loading the required standalone preset JavaScript file, causing a runtime error. This was caught by manual testing on the deployed instance.

Conclusion: AI is a powerful productivity multiplier, but it does not replace domain expertise, manual testing, or critical review. Every AI-generated artifact was validated before integration, and several were corrected or discarded. The final codebase reflects a collaborative human-AI development process where human judgement remained the quality gate.

8 Cloud Deployment & Evidence

The API is fully deployed into a production cloud environment on Render, utilizing a managed PostgreSQL database. To facilitate continuous operation and frictionless evaluation, the

deployment integrates an idempotent automatic seeding script (`scripts.seed_db`) in the web service Start Command. This ensures the PostgreSQL database always retains a functional, analytics-ready "demo" user dashboard upon instantiation.

The successfully deployed endpoints can be explored live via the Swagger UI interface: `https://endurancelife.onrender.com/docs`.

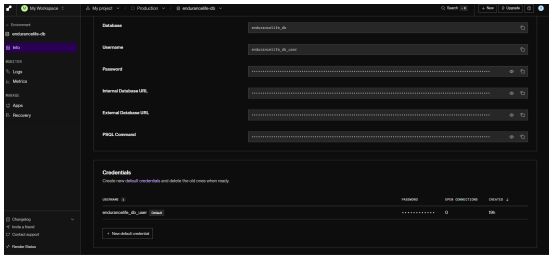

Figure 1: PostgreSQL Cloud Database Instance

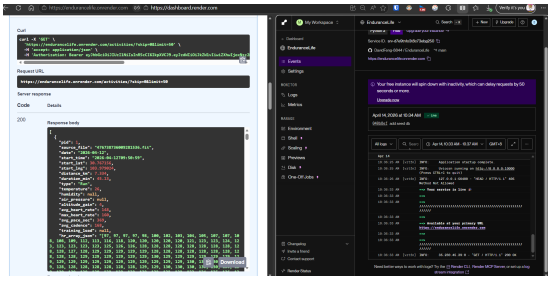

Figure 2: API Server Logs & Swagger UI Integration

9 Conclusion

EnduranceLife demonstrates a complete RESTful API lifecycle: domain modelling, CRUD operations, file upload handling, external API integration, analytics computation, comprehensive testing, and cloud deployment. The technology choices — Python, FastAPI, SQLAlchemy, PostgreSQL — form a cohesive stack that balances development speed with production readiness. The dual-database strategy and batch-optimized data pipelines address the practical challenges of deploying a data-intensive application to a free-tier cloud platform.